



# AI for Phenome × DNA × Health Intelligence

**BDI Young Innovator Hackathon**

Building Data-Driven Systems for Next-Generation Healthcare & Intelligent Living

Teerapong Panboonyuen, Ph.D. (P'Kao)

teerapong.pa@chula.ac.th

<https://github.com/kaopanboonyuen/BDIKKU>



# 3-Hour Workshop Roadmap



0:00–0:20

## Part 0: Foundations

DNA, Phenome & Health AI concepts  
The Big Picture ML Pipeline



0:20–1:20

## Problem 1: DNA Classification

k-mer features · PCA/UMAP · XGBoost  
LASSO selection · SHAP interpretability



1:20–1:30

## Break

Rest, recharge, discuss ideas!



1:30–2:50

## Problem 2: Heart Attack Risk

Clinical EDA · SMOTE · Cox Survival  
Model comparison · SHAP waterfall



2:50–3:00

## Scoreboard & Wrap-Up

Results, prizes & next steps



# 0.1 — What is DNA?

## The Alphabet of Life

DNA (Deoxyribonucleic Acid) carries genetic instructions for all living organisms. It uses only 4 nucleotide bases — a remarkably simple alphabet for the complexity of life.



**Adenine**  
pairs with T



**Thymine**  
pairs with A



**Cytosine**  
pairs with G

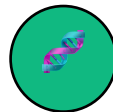


**Guanine**  
pairs with C

### Example DNA Sequence:

ATGCCC CAA CTA AAT ACC GCC GTA TGA CCC...

## From DNA → Gene → Protein → Trait



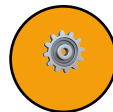
### DNA

The blueprint  
(sequence of ATCG)



### mRNA

Transcribed copy  
(messenger RNA)



### Protein

Translated product  
(amino acid chain)

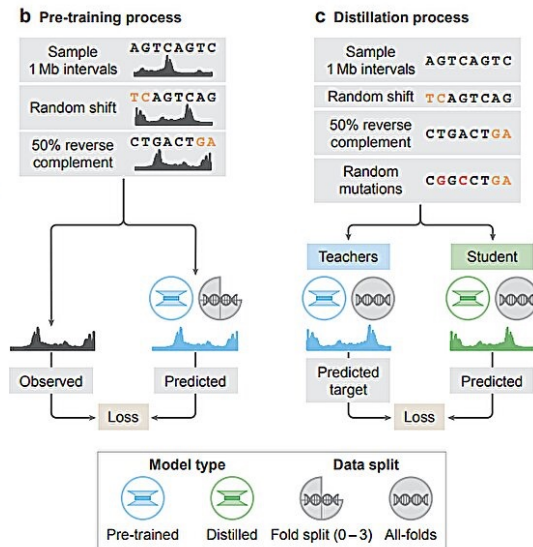
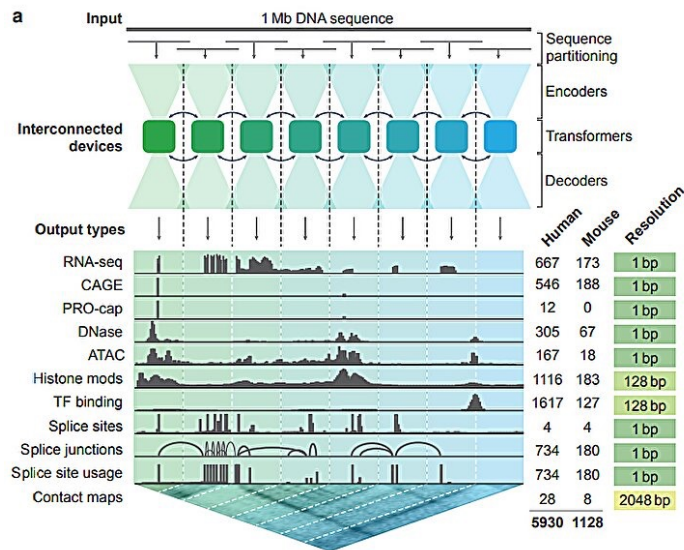
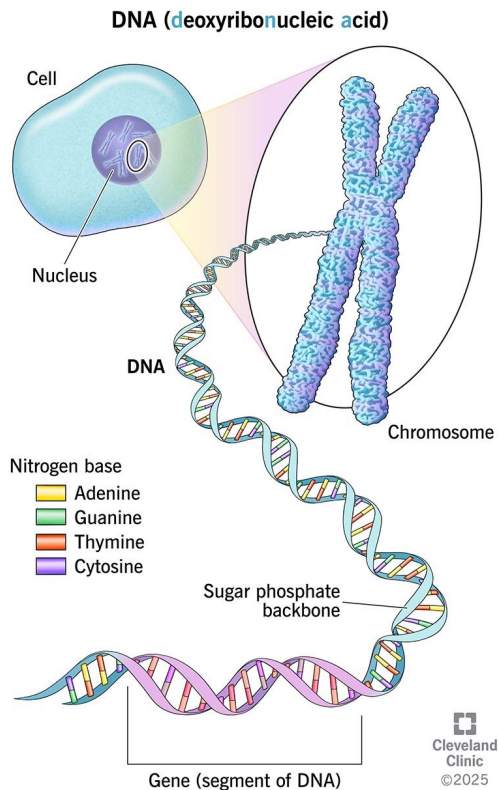


### Trait / Disease

Observable outcome  
(phenotype)



# 0.1 — What is DNA?



Credit:

<https://my.clevelandclinic.org/health/body/dna>,

<https://phys.org/news/2025-09-massive-datasets-generated-powering-latest.html>



## 0.2 — What is the Phenome?



### Genome

All DNA (~3 billion base pairs in humans)



### Transcriptome

All RNA molecules expressed from the genome



### Proteome

All proteins produced in a cell/organism



### Phenome ★

The COMPLETE SET of observable characteristics: height, blood pressure, disease status, lab values



#### Key Insight:

The Heart Attack dataset we use in Problem 2 is a perfect example of phenomic data — structured measurements of observable physiological state (age, blood pressure, cholesterol, ECG, max heart rate).

### Phenome in Healthcare — Examples:

#### Vital Signs

Heart rate, Blood pressure, Temperature, SpO<sub>2</sub>

#### Lab Values

Cholesterol, Blood sugar, Creatinine, HbA1c

#### Imaging

ECG features, X-ray metrics, MRI measurements

#### Outcomes

Diagnosis code, Hospitalization, Survival time



## 0.3 — Why AI for Health?

Traditional medicine relies on population averages. AI enables precision medicine — treating each patient based on their unique genomic + phenomic profile.



### GENOMICS

- ▶ DNA sequences
- ▶ SNP variants
- ▶ Gene expression
- ▶ Epigenomics

+



### PHENOMICS

- ▶ Clinical measures
- ▶ Lab values
- ▶ EMR records
- ▶ Biosignals

+



### LONGITUDINAL

- ▶ Time-series data
- ▶ Survival curves
- ▶ Follow-up visits
- ▶ Wearable data



AI/ML Models → Clinical Decision Support



## 0.4 — The ML Pipeline (Big Picture)



### Key Mathematical Notation:

$$\mathbf{X} \in \mathbb{R}^{n \times d}$$

Feature matrix —  $n$  samples  $\times$   $d$  features  
(the input table)

$$\mathbf{y} \in \{0, 1, \dots, K\}^n$$

Target labels — what we want to predict

$$\mathbf{f}: \mathbf{X} \rightarrow \hat{\mathbf{y}}$$

The model — learns to map features to  
predictions

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$$

Loss function — what the model  
minimizes during training



# Problem 1 — DNA Sequence Classification

## The Challenge:

Given a raw DNA sequence string → predict which gene family it belongs to (Class 0–6). This is a multi-class classification task on biological text data.

0

**G-Protein Coupled**

Signal transduction

1

**Tyrosine Kinase**

Cell signaling & growth

2

**Tyr. Phosphatase**

Opposing kinase activity

3

**Synthetase**

Amino acid activation

4

**Synthase**

Biochemical synthesis

5

**Ion Channel**

Membrane potential

6

**Transcription Factor**

Gene regulation

## Datasets:



Human (Homo sapiens)



Chimpanzee (Pan troglodytes)



Dog (Canis lupus familiaris)





## 1.2 — Feature Engineering: k-mer Frequency

A *k*-mer is any contiguous substring of length *k* from a DNA sequence.

Sequence:

**A T G C C C C A A C T**

k=3 mers:

ATG

TGC

GCC

CCC

CCA

CAA

AAC

ACT

### k-mer Size Comparison:

| k   | Features ( $4^k$ ) | Notes                            |
|-----|--------------------|----------------------------------|
| 1   | 4                  | Base frequencies only            |
| 2   | 16                 | Dinucleotide patterns            |
| 3   | 64                 | Codon-level (amino acids)        |
| 6 ★ | 4,096              | <b>Our choice — best balance</b> |
| 8   | 65,536             | Very high-dimensional            |

### Python: k-mer extraction

```
def get_kmers(sequence, k=6):  
    seq = sequence.lower()  
    return " ".join([  
        seq[i:i+k]  
        for i in range(len(seq)-k+1)  
    ])
```

# Example output (k=6):

# "atgccc tgccca gcccaa ..."

✅ Result: Feature matrix  $X \in \mathbb{R}^{n \times 4096}$  — each sequence becomes a vector of 4,096 k-mer frequencies. Now any ML model can use it!



## 1.3 — Dimensionality Reduction: PCA & UMAP

### PCA

### Principal Component Analysis

**How it works:** Finds directions of maximum variance in high-dimensional data. Projects 4096 k-mer dimensions down to 2D for visualization.

#### Key formula:

$Z = X \cdot W$  where  $W$  = top-k eigenvectors of covariance matrix

#### Explained Variance Ratio (EVR):

$EVR_i = \lambda_i / \sum \lambda_j$  (how much info each PC captures)



**Score plot:** shows how many PCs capture 90% of variance. In our DNA data, ~15-20 PCs are enough!



**PCA result:** Gene families show clear clusters in 2D — the 7 classes ARE separable from k-mer patterns.

### U

### UMAP (Uniform Manifold Approx.)

**How it works:** Non-linear method that preserves local neighborhood structure. Often reveals tighter cluster separation than PCA for biological data.

#### PCA vs UMAP at a glance:

| Aspect        | PCA                | UMAP                  |
|---------------|--------------------|-----------------------|
| Linear?       | ✓ Yes              | ✗ No (non-linear)     |
| Speed         | ⚡ Very fast        | 🕒 Slower              |
| Clusters      | Good               | Excellent             |
| Interpretable | ✓ EVR tells story  | ✗ Axes arbitrary      |
| Use for       | Quick baseline vis | Final cluster insight |

## 1.4 — Feature Selection: LASSO (L1 Regularization)

With 4,096 k-mer features, many are redundant. LASSO helps us keep only the informative ones.

### LASSO Objective Function:

$$\mathcal{L}_{\text{LASSO}}(\beta) = (1/n) \sum (\mathbf{y}_i - \hat{\mathbf{y}}_i)^2 + \lambda \sum |\beta_j|$$

MSE loss + L1 penalty forces  $\beta_j \rightarrow 0$  for unimportant features

### LASSO (L1)

$$\lambda \|\beta\|_1$$

Sets many weights to ZERO  
→ feature selection

### Ridge (L2)

$$\lambda \|\beta\|_2^2$$

Shrinks weights but keeps  
ALL features

### Elastic Net

$$\alpha \|\beta\|_1 + (1-\alpha) \|\beta\|_2^2$$

Best of both worlds  
Correlated features together



### Regularization Path Intuition:

As  $\lambda$  increases (C decreases): more features are forced to zero → simpler, less overfitted model.

C=0.001  
Few features  
High bias



C=0.01  
Some features  
Balanced



C=0.1 ★  
More features  
Good accuracy



C=1.0  
Most features  
May overfit



C=10  
All features  
Overfitting

## 1.4 — Feature Selection: LASSO (L1 Regularization)

*With 4,096 k-mer features, many are redundant. LASSO helps us keep only the informative ones.*

$$L_{\text{LASSO}}(\beta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^d |\beta_j|$$



# 1.5 — XGBoost: The Gradient Boosting Champion

## How XGBoost Works:

1

Start with initial prediction (e.g. mean)

2

Calculate residuals (errors) from current prediction

3

Train a new decision tree to predict the residuals

4

Add tree × learning rate to model ensemble

5

Repeat 300+ times — each tree corrects previous errors

## The XGBoost Math:

### Objective at round $t$ :

$$\mathcal{L}(t) = \sum \ell(y_i, \hat{y}(t-1) + f_t(x_i)) + \Omega(f_t)$$

where  $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$  (regularization)

### Optimal leaf weight:

$$w_j^* = -\sum g_i / (\sum h_i + \lambda)$$

$g_i$  = gradient,  $h_i$  = hessian of loss function

## Key Hyperparameters:

| Parameter        | Value | Tip                             |
|------------------|-------|---------------------------------|
| n_estimators     | 300   | No. of trees (use early stop)   |
| max_depth        | 6     | Tree depth — 3-8 typical        |
| learning_rate    | 0.1   | Shrinkage per step              |
| subsample        | 0.8   | Row sampling (prevents overfit) |
| colsample_bytree | 0.8   | Feature sampling                |



# 1.7 — SHAP: Explaining AI Decisions

## In Healthcare:

A black-box model saying "60% chance of disease" is NOT enough. Clinicians need to know WHY. SHAP provides rigorous, mathematically-grounded explanations rooted in game theory.

## Shapley Value Formula:

$$\phi_j = \sum_{S \subseteq F, j \in S} \frac{(|S|-1)! (|F|-|S|)!}{|F|!} \times [f(S \cup \{j\}) - f(S)]$$

= Average contribution of feature  $j$  across ALL possible feature subsets

## How to Read the SHAP Beeswarm Plot:



### X-axis

SHAP value — positive = pushes prediction UP (more risk), negative = pushes DOWN (less risk)



### Each dot

One sample (patient or DNA sequence). Wider spread = more variable impact across samples.

## 3 Guaranteed Properties:



### Local Accuracy

SHAP values sum to the model's actual prediction



### Missingness

Zero SHAP if feature has no effect on the output



### Color

Feature value — RED = high value, BLUE = low value. Lets you see direction of effect.



### Rows

Features ranked by mean |SHAP| — top rows are most globally important.



# Problem 2 — Heart Attack Risk Prediction

## 🎯 The Challenge:

Given 13 clinical features of a patient → predict whether they have higher or lower chance of heart attack (binary classification). CVD is the #1 cause of death globally!

## 13 Clinical Features (Dataset Dictionary):

**age** **Cont** Patient age in years (29–77)

**sex** **Bin** 0=Female, 1=Male

**cp** **Cat** Chest pain type (0–3)

**trtbps** **Cont** Resting blood pressure mmHg

**chol** **Cont** Serum cholesterol mg/dL

**thalachh** **Cont** Max heart rate achieved bpm

**oldpeak** **Cont** ST depression index

**caa** **Cat** Major vessels (fluoroscopy)

**thall** **Cat** Thalassemia type (0–3)

**exng** **Bin** Exercise-induced angina

**slp** **Cat** Slope of ST segment

**restecg** **Cat** Resting ECG results

**fbs** **Bin** Fasting blood sugar > 120

## Chest Pain Types (key feature!):

0

### Typical angina

Classic cardiac ischemia chest pain

1

### Atypical angina

Chest pain not typical of angina

2

### Non-anginal pain

Non-cardiac chest pain source

3

### Asymptomatic

No pain — paradoxically HIGHER risk!



## Target: output

0 = Less chance of heart attack | 1 = More chance of heart attack



## 2.2 — Handling Class Imbalance: SMOTE

### ⚠ The Problem:

If 95% of patients are healthy, a model that always predicts "No Risk" gets 95% accuracy — but is COMPLETELY USELESS clinically! We must handle class imbalance carefully.

### Strategies for Imbalanced Classification:



#### Class Weights

Penalize minority class misclassifications more during training

✓ No data modification

⚠ Slight accuracy drop



#### SMOTE

Synthesize new minority samples by interpolating between neighbors

✓ More training data

⚠ May create noise



#### Threshold Tuning

Adjust decision boundary after training (e.g. 0.5 → 0.3)

✓ Simple post-hoc fix

⚠ Needs calibrated probs



#### Focal Loss

Down-weight easy examples, focus learning on hard ones

✓ Native in PyTorch

⚠ Custom loss required

#### SMOTE Formula:

$$x_{\text{new}} = x_i + \lambda \times (x_{\text{nn}} - x_i), \quad \lambda \sim \text{Uniform}(0, 1)$$

$x_{\text{nn}}$  = random nearest neighbor of minority sample  $x_i$

#### Class Weight Formula:

$$wc = n_{\text{total}} / (n_{\text{classes}} \times nc)$$

Ensures each class contributes equally to the loss.



## 2.5 — Model Comparison & Evaluation Metrics

### Evaluation Metrics for Binary Classification:



#### Accuracy

$$(TP+TN) / N$$

Balanced classes — quick overview



#### Precision

$$TP / (TP+FP)$$

When false positives are costly



#### Recall (Sensitivity)

$$TP / (TP+FN)$$

★ **Most critical in medicine! Missing a heart attack (FN) is far worse than false alarm (FP)**



#### F1-Score

$$2 \times P \times R / (P+R)$$

Imbalanced classes — harmonic mean of precision & recall



#### AUC-ROC

Area under ROC curve

Overall discrimination ability across all thresholds



### Our Models Compared:

Logistic Regression → LASSO (L1) → Random Forest → XGBoost ★ → LightGBM → CatBoost → XGBoost + SMOTE

## 2.7 — Survival Analysis: Cox Proportional Hazards

### Key Distinction:

**Classification** tells us IF a patient will have a heart attack. **Survival Analysis** tells us WHEN.

### Cox PH Hazard Function:

$$h(t|x) = h_0(t) \times \exp(\sum \beta_j x_j)$$

$h_0(t)$  = baseline hazard (non-parametric) |  $\exp(\beta_j)$  = hazard ratio for feature  $j$

### Hazard Ratio Interpretation:

**HR > 1** Increases risk (HR=2 → twice as likely)

**HR = 1** No effect on event risk

### Clinical Questions Survival Analysis Answers:



What is the 5-year survival probability for a patient with these features?



Which risk factors ACCELERATE disease progression over time?



How does treatment affect the TIME TO EVENT?



Kaplan-Meier curves: does sex affect survival trajectory?

# Workshop Summary — What You've Mastered

## DNA Fundamentals

Nucleotides · Gene families · Central dogma  
DNA→RNA→Protein

## Feature Engineering

k-mer frequency extraction · GC content ·  
CountVectorizer

## Dimensionality Reduction

PCA scree plot · Explained variance · UMAP  
clustering

## Feature Selection

LASSO L1 regularization · Elastic Net ·  
Regularization path

## Tabular ML

XGBoost · LightGBM · CatBoost · Random Forest  
ensemble

## Imbalanced Learning

Class weights formula · SMOTE synthesis ·  
Threshold tuning

## Interpretability

SHAP global+local · Waterfall plots · Permutation  
importance

## Survival Analysis

Cox PH hazard ratios · Kaplan-Meier curves · Log-  
rank test

## Evaluation

Accuracy · Precision · Recall · F1 · AUC-ROC ·  
Confusion matrix

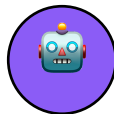


# Hackathon Improvement Ideas & Next Steps



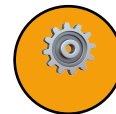
## 1. Better Feature Engineering

Try k=4 or k=5 mers, overlapping windows, GC content per region, dinucleotide patterns



## 2. Deep Learning on DNA

1D-CNN or Transformer directly on raw sequences — no k-mer needed! PyTorch or TensorFlow



## 3. Hyperparameter Tuning

Optuna or Bayesian optimization to find the best XGBoost hyperparameters automatically



## 4. Ensemble Methods

Stack XGBoost + LightGBM + Logistic Regression  
→ often beats any single model



## 5. Cross-Species Transfer

Train on Human + Chimpanzee data, test on Dog  
→ test generalization across species



## 6. Clinical Deployment

Model calibration, fairness audits, decision thresholds, SHAP explanations for doctors



Good Luck in the

# BDI Young Innovator Hackathon!

*"The best way to predict the future is to create it." — P'Kao*

Build → Evaluate → Explain

Data quality > Model complexity

Interpretability = Clinical trust

**Teerapong Panboonyuen, Ph.D. (P'Kao)**

teerapong.pa@chula.ac.th

<https://github.com/kaopanboonyuen/BDIKKU>